SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 30%

Duration : 45 Minutes

QUESTIONS: 2 **Total Marks:20**

Annexure A

Descriptive Written Test

Code of Conduct:

I D.Jeshurun Silas.(192525324) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: D Jeshurun Silas

Annexure A

Descriptive Written Test

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behavior and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.

2. A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem. Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**	**Max Marks**
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

Question 1: Intelligent Agents in a Smart Home Automation System

A hybrid, layered agent is the most effective design for this smart home: reflex rules guard security and safety, a world model fills sensor gaps, goals set targets, and a utility function balances comfort against energy, with learning tuning it over time.

1.1 Introduction

An intelligent agent perceives its environment through sensors and acts on it through actuators to achieve its objectives. The smart home is the agent's environment. Its sensors are motion detectors, temperature sensors, door and window contacts, cameras and a clock. Its actuators are lights, the air conditioner or heater, door locks and the alarm. The task environment is partially observable (a sensor

cannot see every room), dynamic (people and weather keep changing), stochastic and continuous, which is why the choice of agent type matters.

1.2 Types of Intelligent Agents and Their Role in the Smart Home

(a) Simple reflex agent. It selects an action using only the current percept through condition-action rules, with no memory and no model of the world.

- Working: IF motion detected AND time is night THEN switch on the corridor light. IF temperature is above 30°C THEN switch on the air conditioner. IF window sensor is triggered while the alarm is armed THEN sound the alarm.
- Strength: instant response, very cheap, easy to verify, which suits emergencies and security triggers.
- Limitation: it cannot see what it cannot sense. A person sitting still in a room stops triggering the motion sensor, so the lights switch off on them. It also cannot learn.

(b) Model-based reflex agent. It keeps an internal state that tracks the parts of the world it cannot see now, using knowledge of how the world evolves and how its own actions affect it.

- Working: the agent remembers that a person entered the living room and has not left through any door, so it infers the room is still occupied even without motion. It remembers that the AC was switched on five minutes ago and expects the temperature to fall.
- Strength: handles partial observability and filters noisy sensors.
- Limitation: it still only reacts. It has no notion of what it wants to achieve.

(c) Goal-based agent. It uses the model together with an explicit goal and chooses actions by considering which sequence will reach it, using search and planning.

- Working: goal = "house at 24°C when the owner returns at 6 pm". The agent plans to start pre-cooling at 5:30 pm. Goal = "house secured by 11 pm" leads to a plan that locks doors, closes shutters and arms the alarm in order.
- Strength: flexible, since changing the goal changes the behaviour without rewriting rules.
- Limitation: a goal is binary (achieved or not). It cannot say which of two ways of reaching the goal is better.

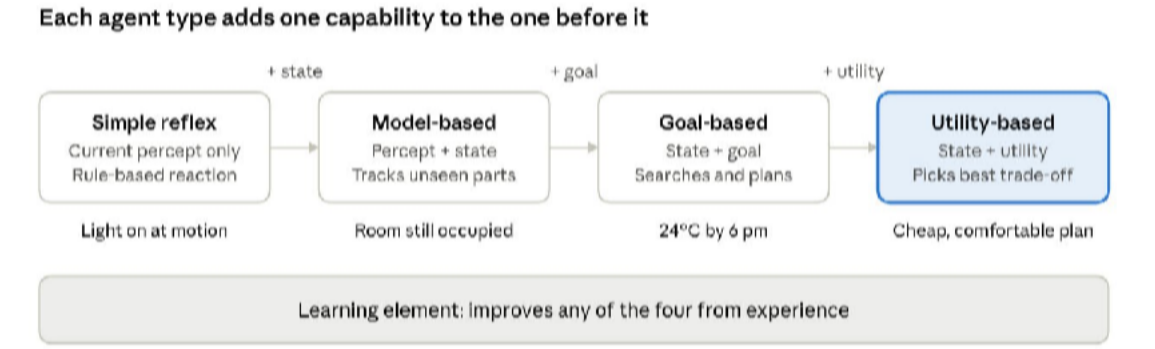
(d) Utility-based agent. It assigns a numeric utility to each possible outcome and picks the action with the highest expected utility, so it can trade off conflicting objectives.

- Working: pre-cooling at full power gives high comfort but high electricity cost. Pre-cooling slowly gives slightly lower comfort and much lower cost. The agent scores both and chooses the better trade-off, also considering tariff at peak hours.
- Strength: resolves conflicts between comfort, energy efficiency and security and handles uncertainty.
- Limitation: designing the utility function and computing expected utility is expensive.

(e) Learning element (added to any of the above). To satisfy the requirement of learning user preferences, a learning component observes the user's manual overrides, such as always lowering the thermostat to 23°C after 8 pm, and updates the rules, the model and the utility weights.

1.3 Comparison of the Agent Types in this System

Agent type	Uses	Smart home example	Handles hidden state	Handles trade-offs	Learns
Simple reflex	Current percept only	Motion switches light on	No	No	No
Model-based	Percept + internal state	Room still occupied, so lights stay on	Yes	No	Limited
Goal-based	Model + goal + planning	Reach 24°C by 6 pm	Yes	No	Possible
Utility-based	Model + utility function	Balance comfort vs cost	Yes	Yes	Yes (weights)



four agent types and the learning element

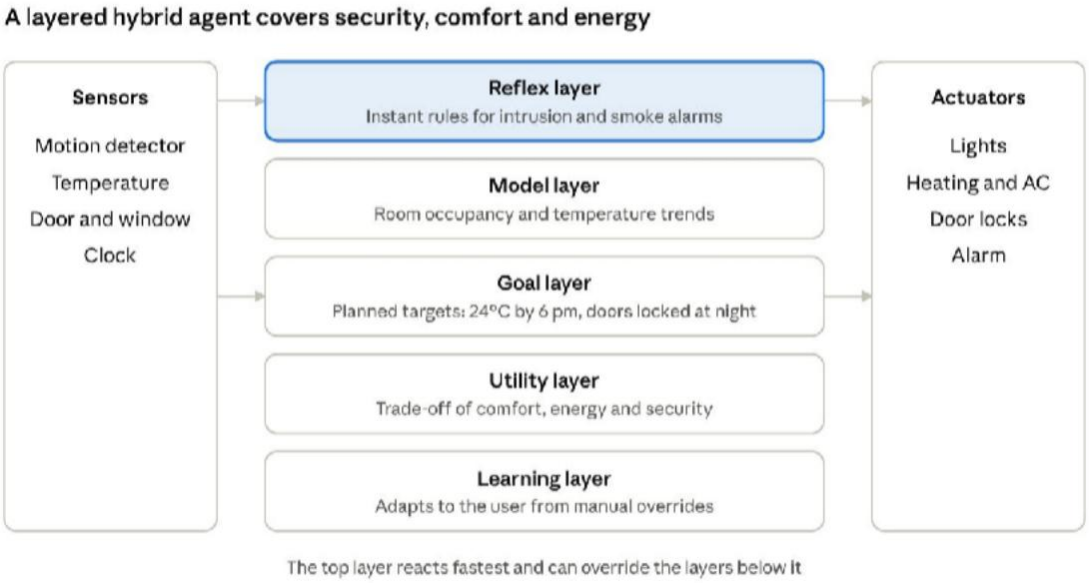
Moving right, each agent type adds one capability, and the learning element can be added to any of them.

1.4 Justification: A Hybrid Layered Agent

No single type satisfies all three requirements, so the most effective design is a hybrid agent in which layers run in parallel and the lower, faster layers can override the upper ones.

1. **Reflex layer for security and safety.** Intrusion, smoke or gas alarms must fire immediately without waiting for planning. A simple reflex rule is fastest and most reliable here.
2. **Model-based layer for comfort inputs.** An internal state of occupancy per room, last-seen times and current temperature trends removes the lights-off-on-a-sitting-person problem and filters faulty readings.
3. **Goal-based layer for scheduled behaviour.** Goals such as "warm bedroom by 10 pm" or "all entry points locked at night" are planned using the model.

4. **Utility-based layer for comfort versus energy.** A utility function such as U = w1 × comfort + w2 × (energy saved) + w3 × security, with weights set by the user, chooses among the candidate plans. Security keeps a very high weight so that energy saving never reduces it.
5. **Learning layer for personalisation.** It adapts thresholds, schedules and weights from the user's behaviour over time.



hybrid smart-home agent · 5 layers

Sensors feed all layers at once. The reflex layer has a direct path to the actuators, so security events never wait for planning.

Outcome against the three requirements: comfort is achieved through the model, goals and learned preferences. Energy efficiency is achieved through utility-based trade-offs, such as pre-cooling at off-peak tariff. Security is achieved through the reflex layer's guaranteed fast response, backed by the goal layer's night-time lock-down plan. This hybrid therefore gives real-time response, prediction, optimisation and adaptation together.

1.5 PEAS Description of the Smart Home Agent

Before choosing an agent type, the task environment is specified using the PEAS framework (Performance measure, Environment, Actuators, Sensors).

PEAS element	Smart home automation system
Performance measure	Occupant comfort, low energy consumption, no security breach, few manual corrections by the user
Environment	Rooms, doors and windows, occupants, outdoor weather, time of day, electricity tariff
PEAS element	Smart home automation system

Actuators	Light switches and dimmers, heater or air conditioner, door locks, shutters, alarm siren, mobile notifications
Sensors	Motion detectors, temperature and humidity sensors, door and window contacts, cameras, light sensors, clock, smart meter

Properties of the environment. The properties below decide how powerful the agent must be.

Property	Value for the smart home	Consequence for the design
Observability	Partially observable	The agent needs an internal state (model) because sensors cannot cover every room at every moment
Determinism	Stochastic	Occupant behaviour and weather are uncertain, so expected utility is more suitable than fixed plans
Episodic or sequential	Sequential	Today's heating decision affects tomorrow's energy bill, so planning is required
Static or dynamic	Dynamic	The world changes while the agent is thinking, so safety responses must be fast
Discrete or continuous	Mixed	Temperature and time are continuous, locks and alarms are discrete
Single or multi-agent	Multi-agent	Several occupants have different preferences, and the power grid tariff acts as another actor

1.6 Agent Programs for Each Agent Type

The pseudocode below shows how the structure of each agent program differs. Each type adds one capability to the previous one.

Simple reflex agent

```
function SIMPLE-REFLEX-HOME-AGENT(percept):
  state <- INTERPRET-INPUT(percept)
  rule <- RULE-MATCH(state, rules)
  return rule.action
```

Model-based reflex agent

```
persistent: state, model, last_action
function MODEL-BASED-HOME-AGENT(percept):
```

```
  state <- UPDATE-STATE(state, last_action, percept, model)
  rule  <- RULE-MATCH(state, rules)
  last_action <- rule.action
  return last_action
```

Goal-based agent

```
function GOAL-BASED-HOME-AGENT(percept, goal):
  state <- UPDATE-STATE(state, last_action, percept, model)
  plan  <- SEARCH-OR-PLAN(state, goal, model)
  return FIRST(plan)
```

Utility-based agent

```
function  UTILITY-BASED-HOME-AGENT(percept):
  state  <- UPDATE-STATE(state, last_action, percept, model)
  actions <- POSSIBLE-ACTIONS(state)
  return the action a in actions with maximum EXPECTED-UTILITY(RESULT(state, a))
```

Proposed hybrid layered agent

```
function HYBRID-HOME-AGENT(percept):
  state <- UPDATE-STATE(state, last_action, percept, model)
   if SAFETY-RULE-MATCHES(state):              // reflex layer, highest priority
      return SAFETY-ACTION(state)
   plans <- PLAN-TO-GOALS(state, goals)        // goal layer
   best  <- plan with maximum EXPECTED-UTILITY // utility layer
   LEARN-FROM-FEEDBACK(state, best, overrides) // learning element
   return FIRST-ACTION(best)
```

The order of the lines is the design decision: safety rules are checked first and can return immediately, while planning and optimisation run only when no emergency exists.

1.7 Scenario-Based Analysis

The table shows which layer of the hybrid agent handles typical situations and why a single agent type would fail on its own.

Scenario	Percepts	Responsible layer	Agent action	Why a single simple type is not enough
Forced entry at night	Window contact opens while the alarm is armed	Reflex	Sound siren, switch on all lights, notify the owner	None needed: a reflex rule is the best answer here
Person reading quietly	No motion for a while, but no door has been used	Model-based	Keep the lights on	A pure reflex agent would switch the lights off on a person who is sitting still
Owner returns home	Clock and learned arrival	Goal-based	Start pre-cooling before arrival so the	A reflex agent only reacts after the owner arrives

Scenario	Percepts	Responsible layer	Agent action	Why a single simple type is not enough
at the usual time	time		house is 24°C	and finds a hot house
Electricity price is high in the evening	Tariff signal, current temperature	Utility-based	Delay or soften the cooling, pre-cool earlier at a cheaper tariff	A goal-based agent only cares whether 24°C is reached, not at what cost
User repeatedly lowers the thermostat after 8 pm	Manual overrides	Learnin g element	Update the preferred temperature for that time	Fixed rules and fixed utility weights never improve
A temperature sensor fails	One sensor reads an impossible value	Model-based	Cross-check with neighbouring sensors and the expected trend, ignore the faulty reading	A reflex agent would act on the wrong value

1.8 Utility-Based Decision: Worked Example

Suppose the owner will arrive at 6 pm and the house is warm at 5 pm. The agent has three candidate actions. Each outcome is scored from 0 to 1 on comfort, energy saving and security. The user's weights are 0.5 for comfort, 0.3 for energy saving and 0.2 for security.

$$U(a) = w_1 \cdot \text{comfort}(a) + w_2 \cdot \text{energy saving}(a) + w_3 \cdot \text{security}(a)$$

Candidate action	Comfort	Energy saving	Security	Utility with weights 0.5 / 0.3 / 0.2
A: Full-power cooling from 5 pm	0.95	0.30	1.0	0.475 + 0.090 + 0.200 = 0.765
B: Gradual pre-cooling at off-peak tariff	0.85	0.80	1.0	0.425 + 0.240 + 0.200 = 0.865
C: Switch cooling on only at 6 pm	0.40	1.00	1.0	0.200 + 0.300 + 0.200 = 0.700

The utility-based agent chooses **action B**, because it has the highest expected utility (0.865). A goal-based agent would treat A and B as equally good, since both reach 24°C by 6 pm.

Effect of changing preferences. If the owner raises the importance of saving energy to 0.5 and lowers comfort to 0.3 (security stays 0.2), the utilities become A = 0.635, B = 0.855 and C = 0.820. Action B still wins, but action C moves much closer. This shows how the same agent adapts its behaviour simply by

changing the weights, with no change to its rules. Security is kept at a fixed weight and treated as a constraint: no action that lowers security can be chosen merely to save energy.

1.9 The Learning Agent Architecture

The question requires the system to learn user preferences over time. Any of the four agent types can be wrapped in a learning agent, which has four components.

Component	Role in general	Role in the smart home
Performance element	Selects external actions	The hybrid layers described above that switch lights, set temperature and arm the alarm
Critic	Judges how well the agent is doing against a fixed standard	Measures comfort complaints, manual overrides and the energy bill against the performance measure
Learning element	Improves the performance element using the critic's feedback	Adjusts schedules, temperature set points and utility weights
Problem generator	Suggests exploratory actions that lead to new experience	Occasionally tries a slightly different set point, for example 25°C instead of 24°C, to find out whether the user also accepts it

Simple methods are sufficient: counting the times each set point is overridden at each hour, or updating a stored preference with a moving average. Because learning happens in the learning element and not in the safety rules, a wrong learned preference can never disable the security behaviour.

1.10 Conflict Resolution, Priorities and Design Challenges

When several layers propose different actions, a fixed priority order resolves the conflict:

1. **Safety** (smoke, fire, gas): always highest.
2. **Security** (intrusion, door and window status).
3. **Comfort** (lighting and temperature as the occupants prefer).
4. **Energy efficiency** (cost and consumption).

Important design challenges and how the design answers them:

- **Sensor noise and failure:** the model layer cross-checks readings and falls back to safe defaults.
- **Privacy:** cameras and behaviour logs are sensitive, so data should be processed locally and stored minimally.
- **Conflicting occupants:** the utility function can weight each occupant's preferences and use the person detected in the room.
- **Explainability and control:** the user must be able to see why an action was taken and to override it at any time, which also provides the feedback used for learning.
- **Cost and complexity:** utility computation is expensive, so it runs only for non-emergency decisions and can be limited to a small set of candidate plans.

- **Security of the system itself:** the controller must be protected from hacking, since a compromised home agent is itself a security risk.

1.11 Advantages and Limitations of the Proposed Hybrid Agent

Aspect	Advantages	Limitations
Response time	Safety and security events are handled instantly by reflex rules	Reflex rules cover only the situations they were written for
Comfort	The model and learned preferences keep rooms comfortable even when sensors are blind	Needs some time to learn a new user's habits
Energy	The utility layer finds cheaper ways to reach comfort	Needs tariff and weather information to be available
Maintainability	Each layer can be tested and upgraded separately	More components to integrate than a single-rule system
Reliability	Failure in the utility or learning layer does not disable safety	Layer interactions must be tested carefully

1.12 Summary of Question 1

Simple reflex agents give speed, model-based agents add memory of hidden state, goal-based agents add planning toward targets, and utility-based agents add the ability to trade off conflicting objectives. A smart home needs all four properties together, plus learning. The recommended solution is therefore a layered hybrid agent in which security is guaranteed by reflex rules, comfort is delivered by the model and goals, energy efficiency is achieved by utility optimisation, and a learning element personalises the whole system over time.

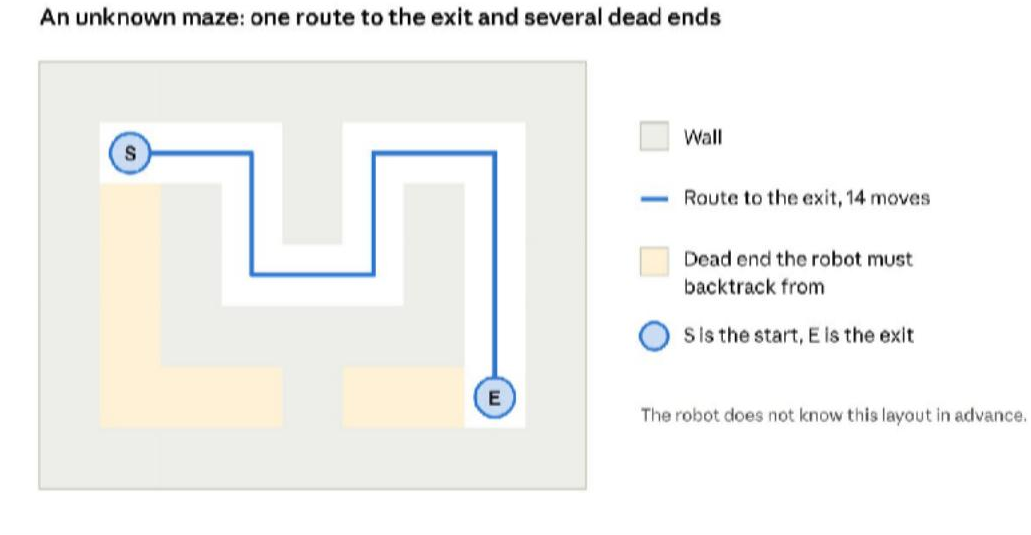
Question 2: Maze Robot, Uninformed Search and Agent Design

A model-based, goal-based agent that explores with Iterative Deepening Search is the best default for an unknown maze, switching to Uniform Cost Search when moves have unequal costs.

2.1 Problem Formulation

- **States:** the robot's position (cell) in the maze.
- **Initial state:** the starting cell. **Goal test:** the cell is the exit.
- **Actions:** move forward, left, right or back to a neighbouring free cell.
- **Transition model:** the cell reached after the move.
- **Path cost:** the cost of each move (1 per step if uniform, or more for turns, slopes or rough terrain).

Because the layout is unknown, the robot has no heuristic that tells it how close the exit is. This is exactly the situation for which uninformed (blind) search strategies are designed: they use only the problem definition and what the sensors reveal while exploring.



example maze with route and dead ends

The robot must find the route while it discovers the walls, and it must backtrack out of every dead end.

2.2 Uniform Cost Search (UCS)

How it works. UCS keeps a priority queue (frontier) ordered by the path cost g(n) from the start. It always expands the cheapest unexpanded node and tests for the goal when a node is selected for expansion, not when it is generated. In the maze, the robot always extends the currently cheapest known route, so cheaper corridors are explored before expensive ones.

Properties. Let C* be the cost of the optimal solution, ε the minimum step cost and b the branching factor.

$$T_{UCS} = O\left(b^{1+\lfloor C^*/\varepsilon \rfloor}\right), \qquad S_{UCS} = O\left(b^{1+\lfloor C^*/\varepsilon \rfloor}\right)$$

- **Advantages:** complete (when every step cost is at least $\varepsilon > 0$) and optimal, so it always returns the cheapest path to the exit. It handles unequal step costs correctly. With equal costs it behaves like Breadth First Search.
- **Disadvantages:** it expands all nodes cheaper than the optimal path, so time can be large. It must store the whole frontier, so space is exponential, which is the main problem for a robot with small memory. It may loop forever on zero-cost cycles, and it needs graph-search bookkeeping to avoid revisiting cells.

2.3 Iterative Deepening Search (IDS)

How it works. IDS runs Depth Limited Search (a depth-first search with a depth cut-off) repeatedly with limits 0, 1, 2, 3 and so on until the goal is found. It combines the low memory of depth-first search with the completeness and shallowest-solution property of breadth-first search. In the maze, the robot explores corridors to depth 1, then restarts to depth 2, and so on, so it never commits to one very deep dead end.

Properties. Let d be the depth of the shallowest goal and m the maximum depth.

$$T_{IDS} = O\left(b^{d}\right), \qquad S_{IDS} = O\left(b\,d\right)$$

- **Advantages:** space is only linear in depth, which suits an embedded robot. It is complete for a finite branching factor and optimal (fewest steps) when every move costs the same.
- **Disadvantages:** states near the top are generated repeatedly, but because most nodes lie at the deepest level this overhead is small (a constant factor of about b/(b-1)). It ignores unequal step costs, so it is not cost-optimal in terrain with different costs. In a physical maze, restarting each iteration means the robot must retrace its route, which costs travel time.

2.4 Comparison of UCS and IDS

Criterion	Uniform Cost Search	Iterative Deepening Search
Expansion order	Lowest path cost g(n) first	Depth-first, depth limit increased each round
Complete	Yes, if step cost $\geq \varepsilon > 0$	Yes, for finite branching factor
Optimal	Yes, minimum total cost	Yes for equal step costs (shallowest goal)
Time complexity	$O(b^{(1+\lfloor C^*/\varepsilon \rfloor)})$	$O(b^d)$
Space complexity	$O(b^{(1+\lfloor C^*/\varepsilon \rfloor)})$, exponential	O(bd), linear
Best when	Step costs differ and memory is available	Memory is limited and costs are equal
Main weakness	Memory growth	Repeated work and ignores costs

2.5 Relating the Problem to Types of Intelligent Agents

The search algorithm is only the tool. The agent design decides how percepts are used and how the next move is chosen.

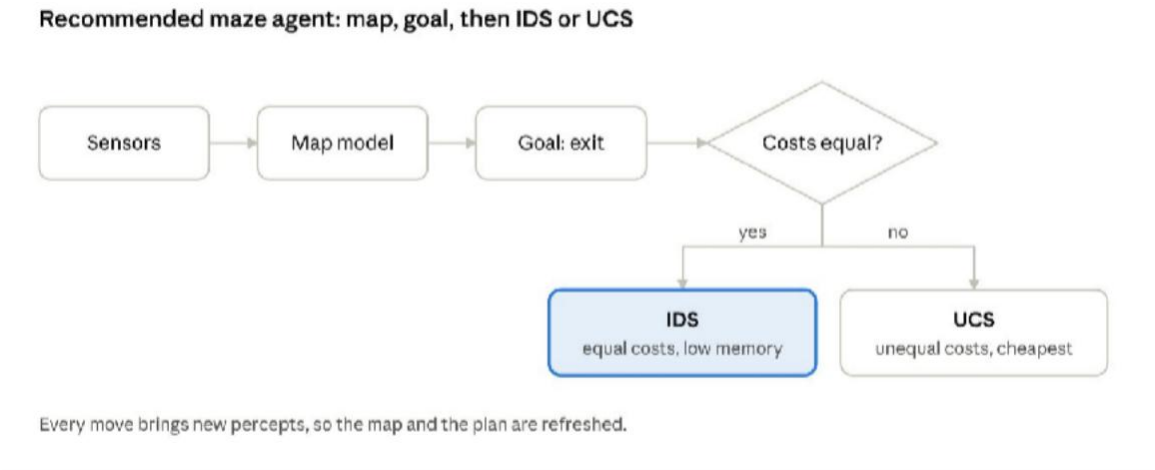
Agent type	Behaviour in the maze	Effect on decision-making
Simple reflex	Rule such as "turn left if wall ahead" or wall-following	Fast and cheap, but it can loop forever and cannot plan or remember visited cells
Model-based	Builds a map of visited cells, walls and dead ends from sensor data	Avoids revisiting cells, copes with partial observability, which is essential in an unknown maze
Goal-based	Goal is "reach the exit", so it searches for a sequence of moves (UCS or IDS)	Decisions are planned and flexible, and the same agent solves any maze
Utility-based	Scores paths by time, battery use, number of turns and risk	Chooses the best path, not just any path, when several routes exist

Thus the agent design determines whether decisions are purely reactive (reflex), informed by memory (model-based), directed by a goal (goal-based) or optimised (utility-based). The search strategy plugs into the goal-based and utility-based layers.

2.6 Recommended Combination and Justification

Recommendation: a model-based, goal-based agent using Iterative Deepening Search, with a utility-aware Uniform Cost Search used when move costs differ.

1. The maze is unknown and only partly observable, so the agent needs the model-based part to build and update a map as sensors reveal walls.
2. The task has a clear goal (the exit), so a goal-based agent can plan a path with search instead of reacting blindly.
3. A mobile robot has limited memory, so IDS with O(bd) space is safer than UCS with exponential space. IDS also returns the shortest path in steps, which is optimal for uniform moves.
4. If turns, slopes or rough surfaces make moves unequal, the utility component switches the planner to UCS (or adds step costs), because only UCS guarantees minimum total cost, provided memory allows.
5. The map from the model-based layer prevents re-exploring known dead ends, which cuts the repeated-work overhead of IDS.



maze robot decision flow · 1 decision

The robot builds a map, aims at the exit, and picks IDS or UCS depending on whether every move costs the same.

Conclusion. For an unknown maze, the model-based, goal-based agent with IDS gives completeness, low memory use and the shortest path. Adding a utility layer with UCS when costs vary gives cost-optimal routes. A simple reflex agent alone is unsuitable because it can loop and cannot plan.

2.7 The Maze as a State Space and Search Tree

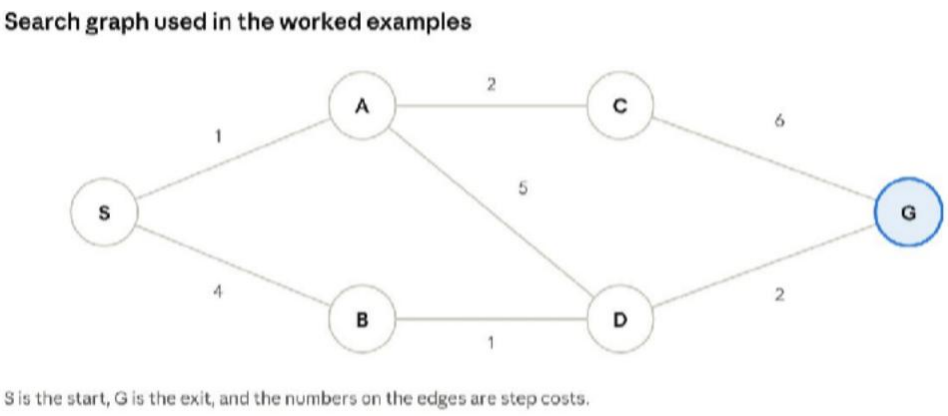
Search treats the maze as a graph. Each free cell is a node, and each possible move is an edge. Starting from the initial cell, the search builds a **search tree** whose root is the start, whose branches are moves, and whose leaves are the cells at the edge of what has been explored (the **frontier**).

- **Branching factor (b):** the number of moves available from a cell. In a corridor maze b is small (often 1 to 3), because walls block most directions. In an open room it can be 4 or more.
- **Depth of the shallowest goal (d):** the fewest moves from the start to the exit.
- **Maximum depth (m):** the longest path that exists in the maze, which can be infinite if the robot is allowed to walk in circles.
- **Repeated states:** the same cell can be reached by different routes. Graph search keeps a record of reached cells so that the robot does not expand the same cell again.

Online versus offline search. In an unknown maze the robot cannot plan the whole route in advance, because it only learns the walls near it. It therefore works as an online agent: it senses, updates its map, plans on the part of the maze known so far, moves, and repeats. The uninformed strategies are applied to the known part of the map, with the nearest unexplored cell or a detected exit acting as the goal.

2.8 Worked Example: UCS and IDS on the Same Search Graph

The small graph below is used to compare the two algorithms step by step. S is the start, G is the exit, and the numbers are the costs of moving between cells (for example, longer or rougher corridors cost more). Edges can be used in both directions, and children are tried in alphabetical order.



sample search graph with step costs

Every route from S to G takes three moves, but the three routes cost 7, 8 and 9.

Uniform Cost Search trace. The goal test is applied when a node is removed from the frontier for expansion.

Step	Node expanded (path cost g)	Frontier after expansion (node with g)	Remark

Step	Node expanded (path cost g)	Frontier after expansion (node with g)	Remark
1	S (0)	A (1), B (4)	Start node expanded
2	A (1)	C (3), B (4), D (6)	Cheapest node A is expanded next
3	C (3)	B (4), D (6), G (9)	G is generated with cost 9 but is not yet chosen
4	B (4)	D (5), G (9)	D is reached more cheaply via B (5 instead of 6), so its cost is updated
5	D (5)	G (7)	G is reached more cheaply via D (7 instead of 9)
6	G (7)	empty	Goal test succeeds

UCS returns the path **S, B, D, G with total cost 7**. Note that it did not accept the first path to G it found (cost 9). It waited until G was the cheapest node on the frontier, which is why UCS is optimal.

Iterative Deepening Search trace. IDS ignores costs and only counts the number of moves.

Depth limit	Nodes visited in order	Result
0	S	Goal not found
1	S, A, B	Goal not found
2	S, A, C, D, B, D	Goal not found, because G is three moves from S
3	S, A, C, G	Goal found on the first path tried

IDS returns the path **S, A, C, G with 3 moves and total cost 9**. It visited 14 nodes in total (1 + 3 + 6 + 4), and cells such as S and A were visited again in each round.

Comparison of the results. Both algorithms found a three-move path, but only UCS found the cheapest one (cost 7 compared with 9). This happens because IDS guarantees the **shallowest** goal, not the **cheapest** goal. The two coincide only when every move costs the same. IDS, however, never held more than one path (at most four nodes) in memory at a time, while UCS had to keep a whole frontier of nodes with their costs.

2.9 Quantifying the Overhead of Repeated Work and the Memory Saving

IDS looks wasteful because it regenerates the upper levels of the tree in every round. The following calculation shows that the overhead is small. For b = 10 and d = 5, the number of nodes generated by IDS is:

$$N_{IDS} = d\,b + (d-1)\,b^{2} + \dots + 1\cdot b^{d} = 50 + 400 + 3000 + 20000 + 100000 = 123450$$

Algorithm (b = 10, d = 5)	Nodes generated	Nodes held in memory
Breadth-first or UCS with equal costs	10 + 100 + 1,000 + 10,000 + 100,000 = 111,110	About 100,000
Algorithm (b = 10, d = 5)	Nodes generated	Nodes held in memory

Iterative deepening (IDS)	123,450	About b × d = 50

IDS generates only about 11% more nodes than breadth-first search, which matches the factor b / (b - 1) = 10 / 9, but it needs roughly 2,000 times less memory. For a robot with a small onboard memory, this is a very favourable trade. These are illustrative values for a uniform tree and are not measured results for any particular maze.

2.10 Position of UCS and IDS Among the Uninformed Strategies

Strategy	Complete	Optimal	Time	Space	Remark for the maze
Breadth-first search	Yes (finite b)	Yes, if all costs are equal	O(b^d)	O(b^d)	Memory grows too quickly for a small robot
Uniform cost search	Yes (step cost ≥ ε > 0)	Yes	O(b^(1+⌊C*/ε⌋))	O(b^(1+⌊C*/ε⌋))	Best when move costs differ
Depth-first search	No (can loop or go infinitely deep)	No	O(b^m)	O(bm)	Can wander into a very deep dead end
Depth-limited search	No, if the limit is below d	No	O(b^l)	O(bl)	Needs a good guess of the limit l
Iterative deepening search	Yes (finite b)	Yes, if all costs are equal	O(b^d)	O(bd)	Best default when memory is limited
Bidirectional search	Yes	Yes, if all costs are equal	O(b^(d/2))	O(b^(d/2))	Not applicable here, because the exit location is not known in advance

2.11 Agent Programs for the Maze Robot

The search algorithms are used inside the agent program. The core of the recommended agent is shown first, followed by the two search procedures.

Recommended model-based, goal-based maze agent

```
persistent: map, position, plan
function MAZE-AGENT(percept):
  map <- UPDATE-MAP(map, position, percept)        // model-based part
  if EXIT-SEEN(map) and position is EXIT: return STOP
```

```
  if plan is empty or plan is no longer valid:
    goal <- EXIT if known else NEAREST-UNEXPLORED-CELL(map)
    strategy <- IDS if step costs are equal else UCS
    plan <- SEARCH(map, position, goal, strategy)  // goal-based part
  return POP-FIRST-MOVE(plan)
```

Iterative deepening search

```
function ITERATIVE-DEEPENING-SEARCH(problem):
  for depth = 0 to infinity:
    result <- DEPTH-LIMITED-SEARCH(problem, depth)
    if result is not cutoff: return result
```

Uniform cost search

```
function UNIFORM-COST-SEARCH(problem):
  node <- NODE(problem.INITIAL, path-cost = 0)
  frontier <- priority queue ordered by path-cost, containing node
  reached <- {problem.INITIAL: node}
  while frontier is not empty:
    node <- POP(frontier)
    if problem.IS-GOAL(node.STATE): return node
    for each child in EXPAND(problem, node):
      s <- child.STATE
      if s not in reached or child.PATH-COST < reached[s].PATH-COST:
        reached[s] <- child
        add child to frontier
  return failure
```

2.12 How Agent Design Affects Decision-Making in the Maze

The same maze situation leads to very different decisions depending on the agent type.

Situation	Simple reflex	Model-based	Goal-based	Utility-based
Robot reaches a dead end	Turns back by a fixed rule, with no memory that the branch is closed	Marks the branch as a dead end on its map and never enters it again	Replans to the nearest unexplored cell	Replans and also weighs the cost of walking back
The route forms a loop	May circle forever	Recognises visited cells and breaks the loop	Excludes reached cells during search	Same as goal-based, with costs
Fork with two corridors	Picks by a fixed rule such as always turning left	Records both options on the map	Searches both with IDS or UCS in a systematic order	Compares expected total cost of each option
One corridor is rough and	Does not notice	Records the terrain if it has a sensor	Ignores cost unless UCS is used	Prefers the cheaper corridor even if it is

Situation	Simple reflex	Model-based	Goal-based	Utility-based
drains the battery				longer
The exit becomes visible	Moves toward it only if a rule exists	Updates the map and the goal	Plans the path straight to the exit	Chooses the best of several known routes to the exit

Therefore agent design determines four things: what the robot **remembers** (the model), what it **wants** (the goal), how it **compares** alternatives (the utility), and how it **adapts** (learning). The search algorithm only becomes useful when the agent has a map and a goal to search with.

2.13 Practical Considerations and Limitations

- **Physical cost of backtracking:** unlike a computer program, the robot must physically travel back to a previous junction. Using the stored map to plan the return route reduces this cost.
- **Localisation and sensor noise:** wheel slippage and noisy range sensors can make the map inaccurate. The model-based layer should correct its position estimate using landmarks and repeated readings.
- **Large mazes:** all uninformed searches have exponential worst-case time. If extra information becomes available, such as the direction of the exit or the straight-line distance to it, an informed method such as A* with a heuristic would explore far fewer cells. This is a different category of algorithm and is outside the scope of this question.
- **Dynamic maze:** if walls or obstacles can move, previous plans can become invalid, so the agent must check its plan after every move and replan when necessary.
- **Time limits:** the robot must stop searching at some point and act. Iterative deepening is convenient here, because it always has a current best-known exploration state and can be stopped and resumed.

2.14 Summary of Question 2

UCS expands the cheapest node first. It is complete and optimal, but its time and memory grow as $O(b^{(1+\lfloor C^*/\varepsilon \rfloor)})$. IDS repeats depth-limited searches with a growing limit. It is complete and finds the shallowest goal in $O(b^d)$ time while using only $O(bd)$ memory. The agent type decides how these algorithms are used: a simple reflex agent cannot plan or remember, a model-based agent builds the map that makes search possible in an unknown maze, a goal-based agent uses search to reach the exit, and a utility-based agent chooses the best route among several. The best combination for the given problem is a model-based, goal-based agent with IDS as the default strategy and UCS when step costs are unequal.

Overall Conclusion

Both questions lead to the same principle: an agent's intelligence comes from how it combines memory, goals and preferences, and a search or decision method is only as useful as the agent design around it.

- In the **smart home**, the environment is partially observable, dynamic and has conflicting objectives. No single agent type is enough, so a layered hybrid agent with reflex, model-based,

goal-based and utility-based layers plus a learning element gives fast security responses, comfortable rooms and lower energy use.

- In the **maze**, the layout is unknown and memory is limited. A model-based, goal-based agent using Iterative Deepening Search gives a complete search with linear memory and the shallowest path, and switching to Uniform Cost Search when move costs differ gives the cheapest path.

The worked numerical examples confirm the theory. The utility calculation selected the gradual off-peak pre-cooling option (0.865), and the trace on the sample graph showed UCS finding cost 7 while IDS found cost 9 with the same number of moves.

Mapping to Course Outcome CO1

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively (BL3).

CO1 element	Where it is applied in this report
Intelligent agents	Sections 1.2 to 1.9: four agent types, PEAS analysis, agent programs, learning agent, hybrid design
Uninformed search methods	Sections 2.2 to 2.10: UCS, IDS, complexity analysis and comparison with other uninformed strategies
Solving problems effectively (application, BL3)	Sections 1.7, 1.8, 2.8 and 2.11: scenario analysis, utility calculation, traces on a sample graph and agent programs
Justified design choice	Sections 1.4 and 2.6: hybrid layered agent and the model-based, goal-based agent with IDS (UCS when costs differ)

Glossary of Key Terms

Term	Meaning
Agent	An entity that perceives its environment through sensors and acts on it through actuators
Percept	The agent's input from its sensors at one moment
PEAS	Performance measure, Environment, Actuators, Sensors: a way to specify a task environment
Simple reflex agent	Chooses actions from the current percept using condition-action rules
Model-based agent	Keeps an internal state that represents the parts of the world it cannot currently observe
Term	Meaning

Goal-based agent	Chooses actions that lead toward an explicit goal, usually by search or planning
Utility-based agent	Chooses the action with the highest expected utility, which measures how desirable an outcome is
Learning agent	An agent that improves its behaviour from experience using a critic and a learning element
State space	The set of all states reachable from the initial state by any sequence of actions
Frontier	The set of generated nodes that have not yet been expanded
Branching factor (b)	The maximum number of successors of a node
Depth (d)	The number of moves from the start to the shallowest goal
Path cost g(n)	The total cost of the path from the start node to node n
Completeness	A search algorithm is complete if it finds a solution whenever one exists
Optimality	A search algorithm is optimal if the solution it finds has the lowest cost
Uninformed search	Search that uses only the problem definition and no estimate of the distance to the goal
Uniform Cost Search (UCS)	Expands the node with the lowest path cost first
Iterative Deepening Search (IDS)	Repeats depth-limited search with an increasing depth limit

References

1. Russell, S. and Norvig, P. (2021). *Artificial Intelligence: A Modern Approach*, 4th edition. Pearson. Chapter 2 (Intelligent Agents) and Chapter 3 (Solving Problems by Searching).
2. Korf, R. E. (1985). Depth-first iterative-deepening: An optimal admissible tree search. *Artificial Intelligence*, 27(1), 97-109.
3. Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1, 269-271. (The basis of uniform cost search.)